EVENT-DRIVEN PROGRAMMING

FINAL PROJECT & HACKATHON CHALLENGE

Project Challenge

QR-ATT

Smart School Event Attendance Management System

---

1. Project Concept

Your final project for the **Event-Driven Programming Course** will also serve as your preparation for the upcoming **Hackathon**.

You will first develop your own working version of a **School Event Attendance Management System** individually.

After the Final Examination, you will form a **Hackathon Team of five (5) members**. Your team will use your individual projects, ideas, skills, and experience as the foundation for creating an improved and innovative solution during the Hackathon.

The goal is simple:

> ***Build a working attendance system individually. Then, as a team, transform the basic system into a smarter and more innovative campus solution during the Hackathon.***

PART I — INDIVIDUAL FINAL PROJECT

2. Final Project Objective

Develop a functional mobile or web-based system that allows a school to manage event attendance using **QR codes**. The system must demonstrate your understanding of:

- Event-driven programming
- User interface design
- Database integration
- Authentication
- Authorization
- CRUD operations
- QR code generation/scanning
- Data validation
- Security
- Real-time user interaction

You must be able to explain and demonstrate your own implementation during the Final Examination.

3. Required User Roles

Your system must support at least three roles:

A. Student

The Student can:

- Create/login to an account
- View personal information
- View available events
- Scan an event QR code
- Record attendance
- View personal attendance history

B. Teacher

The Teacher can:

- Login securely
- Create/manage events
- Generate an event QR code
- View attendance records
- Monitor attendance
- View attendance summaries

C. Administrator

The Administrator can:

- Manage users
- Manage events
- View overall attendance
- Manage or correct attendance records
- View system records/reports

4. Minimum Required Features

Your individual project must contain the following:

Authentication

- Login
- Registration
- Logout
- Session handling
- Password protection

User Management

- Student profile
- Teacher profile
- Administrator profile
- Role-based access

Event Management

A teacher or administrator must be able to:

- Create an event
- Edit an event
- View an event
- Close an event

An event should contain information such as:

- Event name
- Date
- Time
- Venue
- Description
- Status

QR Attendance

The system must allow:

1. Teacher/Admin creates an event.
2. System generates a QR code.
3. Student scans the QR code.
4. System validates the event.
5. System records the student's attendance.
6. Student receives confirmation.

Attendance Validation

The system must prevent:

- Duplicate attendance
- Attendance for closed events
- Invalid QR codes
- Unauthorized attendance recording

Attendance Records

The system must record at least:

- Student
- Event
- Date
- Time
- Attendance status

Attendance History

Students must be able to view their own attendance history. Teachers/Admins must be able to view attendance records for their events or the system.

5. Database Requirement

Your application must use a database. At minimum, your database should contain appropriate tables/entities for:

- Users
- Profiles
- Events
- Attendance

You may add additional tables when necessary. Your database must properly implement relationships between records.

6. Security Requirements

Your application must demonstrate basic security. You must implement:

- Authentication
- Role-based authorization
- Database access protection
- Input validation
- Duplicate attendance prevention
- Secure session handling
- Proper logout
- Protected database operations

If using Supabase, appropriate **Row Level Security (RLS)** policies must be implemented.

Important:

Never place passwords, secret keys, service-role keys, or other sensitive credentials directly in your source code or public Git repository.

7. Event-Driven Programming Requirement

Your project must clearly demonstrate event-driven programming. During your presentation, you must be able to identify at least **five (5) important events** in your application.

Examples:

1 User presses Login.

2 QR Scanner detects a QR code.

3 Attendance button is pressed.

4 Event is created.

5 Event status changes.

6 Database record is successfully inserted.

7 Invalid QR code is detected.

8 Duplicate attendance is detected.

9 User logs out.

10 Data is refreshed.

For each example, you should be able to explain:

> ***Event → Handler → Action → Result***

8. User Interface Requirements

Your application should be:

- Clean
- Easy to navigate
- Consistent
- Mobile-friendly or responsive
- Easy for students and teachers to understand

Your system should provide appropriate feedback for:

- Loading
- Success
- Error
- Empty data
- Invalid QR

- Duplicate attendance
- Unauthorized access
- Closed event

9. Individual Final Exam

During the Final Examination, you will demonstrate your own project individually. You must be able to:

1. Explain the purpose of your system.
2. Demonstrate the major features.
3. Explain your database.
4. Explain your event-driven processes.
5. Explain your authentication and authorization.
6. Demonstrate QR attendance.
7. Answer technical questions about your implementation.

Important:

You are expected to understand the code and implementation of your own project. You may be asked to modify a feature, explain a function, identify a database relationship, or troubleshoot a problem during the presentation.

10. Individual Final Project Rubric

Criteria	Points
Core Functionality	25
Authentication & Role-Based Access	15
Database Implementation	15
QR Attendance Process	15
Event-Driven Programming	10
UI/UX	10
Security & Validation	5
Individual Presentation & Defense	5
TOTAL	**100**

PART II — HACKATHON

11. Hackathon Team

After the Final Examination, students will form teams of **5 MEMBERS PER TEAM**. Each member will bring the knowledge and experience gained from their individual final project. The team may:

- Select one individual project as the starting point.
- Combine ideas from different members.
- Rebuild the system.
- Redesign the system.
- Create a new architecture.
- Implement new features.

The team is **not required to simply combine five separate applications**. The objective is to produce **one unified solution**.

12. Hackathon Challenge

Challenge: SMART CAMPUS ATTENDANCE

> ***How can you transform a basic QR attendance system into a smarter, faster, more secure, and more useful campus attendance solution?***

Your team must create a solution that improves the basic attendance process. The final hackathon system must address real problems encountered in school attendance.

13. Hackathon Core Requirement

Your team must retain the basic attendance functionality:

- User authentication
- Student access
- Teacher/Admin access
- Event management
- QR attendance
- Attendance records
- Database integration
- Security

However, your team must go beyond the basic requirements. Simply reproducing the individual final project will not be sufficient for the Hackathon.

14. Hackathon Innovation

Each team must implement **at least three (3) significant improvements**. Possible improvements include:

Smart QR

- Dynamic QR
- Rotating QR
- Time-limited QR
- QR expiration
- Anti-screenshot/reuse mechanism

Smart Attendance

- Late detection
- Attendance status
- Automatic validation
- Location verification
- NFC/RFID integration
- Face verification

Offline Capability

Allow attendance to continue temporarily when the internet connection is unavailable and synchronize the records when the connection returns.

Real-Time Monitoring

Display attendance statistics in real time. Example:

Expected: 100	**Present:** 82	**Late:** 10	**Absent:** 8

Analytics

Provide useful information such as:

- Attendance percentage
- Most attended events
- Attendance by section
- Attendance by program
- Late attendance statistics

- Event attendance trends

Notifications

Notify users about:

- Upcoming events
- Successful attendance
- Late attendance
- Event changes
- Attendance reminders

Administrator Dashboard

Provide a centralized dashboard for monitoring campus-wide attendance.

Other Innovation

Teams may propose their own feature if it clearly improves the attendance process.

15. Hackathon Constraint

The Hackathon is not simply a feature-counting competition. Your team should be able to explain:

PROBLEM	What problem are you solving?
USERS	Who experiences the problem?
SOLUTION	How does your system solve it?
TECHNOLOGY	What technologies did you use?
INNOVATION	What makes your solution different from the basic system?
IMPACT	How can your solution improve school attendance management?

16. Hackathon Team Roles

Each team member should have a clear contribution. Suggested roles:

1. **Project/Team Lead**
2. **Frontend/Mobile Developer**
3. **Backend/Database Developer**
4. **UI/UX & Integration Developer**
5. **QA/Security/Documentation Lead**

One member may perform more than one role. However, **all five members must understand the system**.

17. Hackathon Final Output

Each team must produce:

1. Working Application

The application must be functional and demonstrable.

2. Source Code

The complete source code must be submitted through the designated repository.

3. Database

The database must be properly structured and functional.

4. Documentation

The team must provide:

- System overview
- Architecture
- Database design
- Major features
- Security implementation
- Testing results

5. Presentation

The team must explain:

- Problem
- Solution
- Features
- Technology
- Architecture
- Innovation
- Demonstration

18. Hackathon Presentation

The presentation should focus on the **solution**, not simply on reading documentation.
Recommended sequence:

Segment	Time	Focus
1. Problem	2 minutes	What problem did you identify?
2. Solution	2 minutes	What did you build?
3. Demonstration	5–10 minutes	Show the actual system.
4. Technical Explanation	3 minutes	Explain the architecture, database, and important technologies.
5. Innovation	3 minutes	Show what your team added beyond the basic system.
6. Defense / Q&A	—	The judges may ask any member questions about the project.

19. Hackathon Rubric

Criteria	Points
Functionality & Reliability	20
Innovation & Problem Solving	20
Technical Implementation	15
Event-Driven Programming	10
Database & Security	10
UI/UX	10
Team Integration	5
Presentation & Demonstration	5
Documentation	5
TOTAL	**100**

20. Important Rules

Rule 1 — Individual Work First

Your Final Project must be developed and understood individually.

Rule 2 — Teamwork Later

The Hackathon will be completed by teams of five (5).

Rule 3 — Do Not Simply Copy

The Hackathon output must contain meaningful improvements beyond the individual project.

Rule 4 — Every Member Must Contribute

Each member must have a clear and demonstrable contribution.

Rule 5 — Every Member Must Understand the System

During the Hackathon presentation, any member may be asked technical questions.

Rule 6 — Your Code Must Be Your Own Work

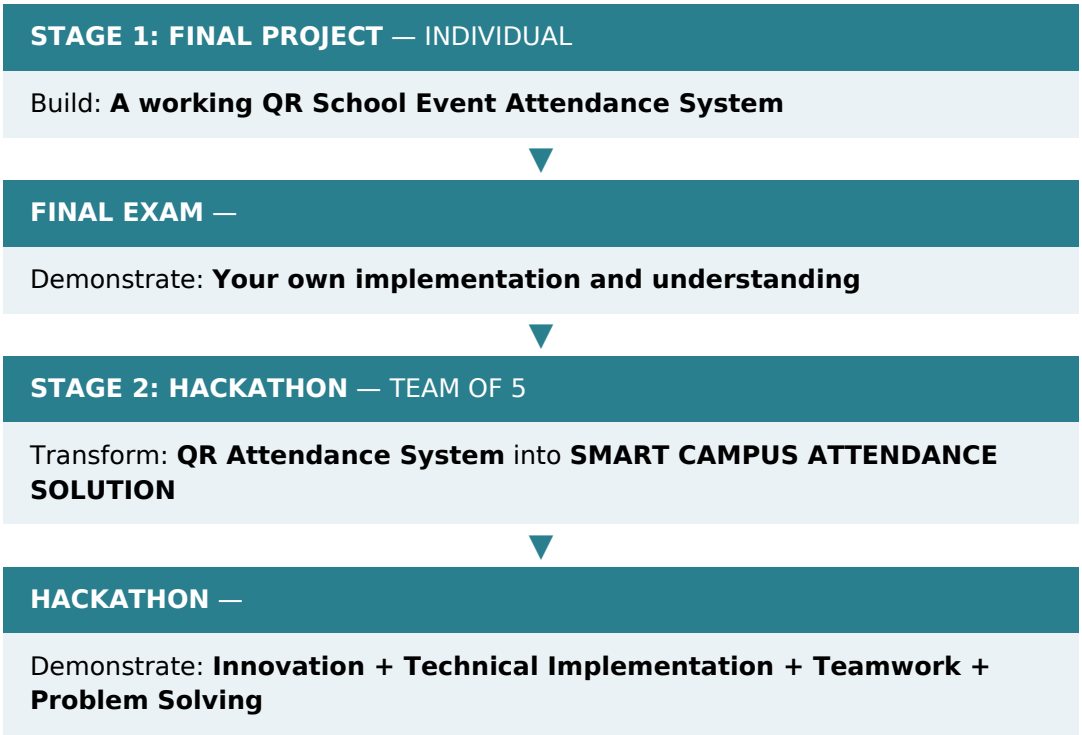
You may use libraries, frameworks, APIs, and documented resources, but you must understand and properly integrate them.

Rule 7 — Protect Your Credentials

Never submit passwords, API secrets, database credentials, or service-role keys to a public repository.

21. The Development Roadmap

You should think of the project as a two-stage development process:



22. Final Message to Students

Your Final Project is not a project that ends after the Final Examination. It is your **starting point for the Hackathon**.

Build your individual project carefully. Learn your code. Understand your database. Practice your presentation. Think about the problems that your system can solve.

After the Final Examination, you will have the opportunity to work with four other developers and transform your individual experience into a larger team solution.

Your challenge is:

> ***BUILD IT INDIVIDUALLY.***
> ***IMPROVE IT AS A TEAM.***
> ***INNOVATE IT FOR THE HACKATHON.***

Good luck, and build something worth demonstrating.